

A Homomorphic Operator Layer for arithmetic that generalizes

Kronecker Embeddings V2 · individual submission · Kunal Sinha

Abstract

We consider embeddings that store mathematical structure — so that the vector of 9 combined with the vector of 9 becomes the vector of 18, and of 81. Storing arithmetic this way is the easy half: a spectral (Fourier) number code turns addition and multiplication into a single fixed operation (elementwise complex product) that is exact and needs zero parameters. The mechanism is not new — it is frequency-domain holographic binding (HRR/VSA; Plate 1995, Kanerva 2009), with the log-domain for multiplication (the classical log-number system), and Fourier number features are known to emerge in trained networks (grokking; Nanda et al. 2023). Our contribution is a **framing and a controlled result**: cast the construction as a *mathematical intermediate representation (IR)* with **explicit operators**, and ask whether making the operation an *architectural primitive* — rather than hoping a network learns it from features — is what enables **magnitude extrapolation**. In a controlled test (same representation, same inputs, same answer-only supervision, varying only the operator), a model with the operator built in extrapolates to unseen magnitudes while the identical model with a learned head instead does not, and a NAC/NALU baseline (Trask et al. 2018) learns the operation only approximately. We then characterise, via representation theory, *which factors* admit such an operator. The evidence supports "the operator, not the representation, is what generalizes" as a tested claim in this controlled setting, not as a universal law.

1. Motivation: a mathematical IR

The usual framing asks whether an embedding can itself perform mathematics (`embedding` $\rightarrow$ `answer`). A compiler analogy suggests a stronger one. A compiler never treats `9 + 9` as raw bits from the start; it keeps progressively lower-level representations that *preserve semantics*. At the intermediate-representation (IR) level, `ADD(9, 9)` is not two bit-strings — it carries the meaning "integer addition of these two values," and only later is that lowered to machine ops and bits. Crucially, `ADD` there is an *explicit operator*, not something inferred from the operands. This reframes the goal: build a **mathematical IR inside a neural representation** — a latent number representation plus explicit operators — so computation becomes compositional in that latent space (`M(9)  $\odot$  M(9)  $\rightarrow$  M(18)` , then `M(18)  $\odot$  M(9)  $\rightarrow$  M(162)`). The pipeline is exactly a compiler's: natural language $\rightarrow$ neural encoder $\rightarrow$ mathematical IR $\rightarrow$ transformations $\rightarrow$ result. More precisely, the tightest correspondence is a compiler's *front-end* $\rightarrow$ *IR* $\rightarrow$ *constant-folding* path: the number codes and explicit operators are the IR (`M(9)  $\odot$  M(9)  $\rightarrow$  M(18)`), analogous to `%c = add i32 9, 9`), and evaluating the operator is exactly constant folding (`%c = 18`). The Operator Layer is thus an IR-level *evaluator* rather than a code generator — the natural home for exact arithmetic in representation space, and in sharp contrast to `embedding  $\rightarrow$  answer` , which has no IR at all. The remainder of this report constructs that IR and its operators, and tests the pipeline.

2. Spectral number codes

A number `n` is represented by a bank of complex phasors:

- additive code $a(n) = [e^{i \omega_k n}]_k$ — for $+$, $-$
- logarithmic code $m(n) = [e^{i v_k \ln n}]_k$ — for $\times$, $\div$

Both are deterministic and invertible: an integer is recovered by a matched filter that scores candidate values against the code's phases. Two homomorphisms follow directly from the phase, because shifting a number rotates every phasor and a logarithm turns products into sums:

$$a(x + y) = a(x) \odot a(y) \qquad m(x \cdot y) = m(x) \odot m(y)$$

where $\odot$ is the elementwise complex product. Subtraction and division use the conjugate. The operation is therefore a fixed, parameter-free **bind**, and the "embedding of 9 combined with the embedding of 9" literally decodes to 18 (additive band) and 81 (log band).

Why Fourier (a unit-circle Z-transform). The additive code evaluates the number on the **unit circle**, $z = e^{i\omega}$; the log code is its multiplicative analogue (a Mellin / log-Fourier transform, $n^{i\omega}$). The Z-transform is the general family — it evaluates at any complex $z = r \cdot e^{i\omega}$ — and the homomorphism $z^{a+b} = z^a \cdot z^b$ holds anywhere in the plane. But off the unit circle ($r \neq 1$) the magnitude r^n grows or decays exponentially with n , which is numerically unstable and un-decodable. The unit-circle ($r = 1$) restriction keeps every component at magnitude 1 while preserving the exact "add $\rightarrow$ phase-rotation" homomorphism. It is also canonical, not arbitrary: the characters of the group $(\mathbb{Z}, +)$ live exactly on the unit circle (its Pontryagin dual), so Fourier is *the* basis in which addition diagonalizes — the general point of §6.

Exactness. With no trainable parameters, addition and subtraction decode exactly across the tested range ($a, b < 4000$), multiplication and division decode exactly across the tested product range, and chained additions remain exact to at least 32 terms with no drift.

3. The problem: structure that a model won't use

Exact storage does not imply a model can compute with it and extrapolate. If the spectral features of a and b are handed to a standard multilayer network that must produce $a + b$, the network fits the training range but collapses on larger, unseen magnitudes. This is the core difficulty: the structure is present in the representation, but a generic learned readout does not inherit it.

4. The Homomorphic Operator Layer

The layer separates what should be learned from what is fixed:

1. **Encode.** Numbers become spectral codes; the words of the query become ordinary learned token embeddings.
2. **Gate.** A small learned gate maps the *phrasing* of the query ("plus"/"add"/"sum" $\rightarrow +$) to one of the operations. This is the only trained component, and it generalizes across phrasings.
3. **Bind and decode.** The selected exact bind combines the two number-codes and a fixed decoder reads out the integer. This step has no parameters and is valid at any magnitude.

How a query becomes an answer. Two mechanisms are cleanly separated — *which operation* is learned from language, while the *operation itself* is a fixed homomorphism. A reviewer's natural question, "how does the model know to add, and how does it check against $\text{Rep}(18)$?", is answered stage by stage:

```

"9 plus 9"
| gate: the word "plus" → operation ADD          (the only learned step)
▼
Rep(9), Rep(9)                                  (deterministic encode)
| bind: Rep(9) ⊙ Rep(9) = Rep(18)                (fixed homomorphism, 0 params)
▼
Rep(18)
| decode: matched filter vs the codebook {Rep(n)} → argmax = 18
▼
18

```

The model never infers "add" from the operands and never *searches* for the answer: the bind *produces* `Rep(18)` by construction, and the only comparison to `Rep(18)` is the decode — a nearest-representation readout that turns the vector back into the integer.

The network thus learns only *what the query asks for*; it never learns arithmetic, which is fixed and exact. That is precisely what converts "memorize a range" into "compute for any magnitude."

5. Controlled experiment: is it the operator or the representation?

A weak comparison would pit the operator layer against an arbitrary baseline and declare victory; that proves little. The claim we actually want to test is causal — *does making the operation an architectural primitive, rather than a learned map, cause magnitude extrapolation?* — so the experiment holds **everything else fixed** and varies only that (`experiments/end2end.py`).

Setup. Queries "`<a> <word> <b>`" over `+` and `-` (kept to two operations so a *differentiable* soft matched-filter decode over a small answer range is tractable); numbers `0. . 199` ; train on pairs with `max(a, b) < 120` , test on `max(a, b) ≥ 120` . Crucially, **supervision is the answer only** — no operation labels — so any op-selection must be learned from the answer signal, and every model sees the same inputs (operand codes + an op-word embedding), the same output space, and the same loss. Three seeds.

model (answer-only supervision)	train	unseen magnitude
HOL — fixed operator (complex-product bind)	1.00	1.00
learned bind, shared decode — clean ablation	1.00	0.00
learned answer-classifier — weaker ablation	1.00	0.00
NAC / NALU (Trask et al. 2018) on scalar values	0.13	0.05 (within ±1: 0.10)

The **clean ablation** is the load-bearing comparison. It keeps the *same* operand codes and the *same* differentiable matched-filter decode over the *same* candidate codebook, and replaces **only** the fixed complex-product bind with a learned map on the same codes. Because the decode ranges over the whole codebook, unseen-magnitude answers stay fully reachable — so its failure is not a label-coverage artifact. It fits training (1.00) and still collapses out of distribution (0.00), while the fixed operator holds at 1.00. That isolates the cause: the *exactness of the operator*, not the representation or the reachable output set, is what extrapolates. (The weaker "answer-classifier" ablation reaches the same 0.00 but partly for a structural reason — its fixed answer-class head never sees test-range labels in training — so we do not lean on it; it is listed only for completeness.)

Two honest caveats. First, HOL's *only* learned parameters are the small op-word gate, so its train/extrapolation are near-trivial to fit (std 0 across seeds) — the point is not that HOL learns something hard, but that its fixed operator carries exactness to any magnitude where the learned bind cannot. Second, the NAC baseline is not a strawman: it genuinely learns the operation (weights converge to $\approx [1, 1]$ for $+$ and $[1, -1]$ for $-$), but NAC weights only *approach* integer values, so its error is multiplicative and grows with magnitude (mean absolute error $2.1 \rightarrow 4.3$), which exact-integer scoring penalises — precisely the value of an *exact* operator over a learned-approximate one.

This supports — in this controlled setting — that arithmetic extrapolation here is **architectural**: it comes from the operator, not from the features. (The earlier `operator_layer.py` covers all four operations $+$ $-$ $\times$ $\div$ with an op-word gate and is kept as a breadth illustration; the causal claim rests on the clean ablation above.)

6. What are the right factors? A factorization ablation

The structured-token idea `byte  $\otimes$  position` beats an arbitrary learned token ID because it factors identity into meaningful parts. For numbers the sharper question is not "can we copy that?" but *what are the right factors?* Different factorizations serve different jobs: `byte  $\otimes$  position` is right for identity, `digit  $\otimes$  place` for decimal notation and magnitude, and a **character / irreducible-representation basis** (Fourier $e^{i\omega n}$ for $+$, log-Fourier for $\times$) for the *operation* — because in that basis the operation becomes pointwise. The unifying principle is representation-theoretic: the right factors are the characters of the algebraic structure whose operation you want exact and cheap, i.e. the basis in which that operation diagonalizes.

We exhibit this directly (`experiments/factorization.py`) by factoring an integer three ways and giving each the operator natural to it — this measures operator *admissibility*, not a learned-model comparison (that is §5). Each entry is the exact accuracy of that factorization's own operator across magnitude bands ($n = 60$ – 120 random pairs per band):

factorization	natural operator	exact across magnitudes?	admits an exact operator?
Fourier characters <code>$e^{i\omega n}$</code>	pointwise complex product (0 params)	100% (bands to 2×10^5 , $n=60$ – 120)	yes — pointwise, bounded range
<code>digit $\otimes$ place</code> (base 10)	digit-wise add + carry	100%, unbounded	yes — sequential (carry)
arbitrary learned ID	(none; a learned MLP must memorise)	train 100%, unseen 0%	no

Two conclusions follow. First, **a factorization is useful for an operation iff it admits a simple exact operator, whatever its ability to represent the number**: all three can represent any integer, yet only the two with an exact operator compute it across magnitudes; the arbitrary learned ID has *no* operator, so a model can only memorise and then fails on unseen inputs (its 0% is a memorisation failure; with a fixed answer-class head, test-range answers are also unseen labels — the clean learned comparison is the §5 ablation). This is the Kronecker lesson sharpened — structure helps because it *admits an operator*. Second, among structured factors there is a **range $\leftrightarrow$ operation tradeoff**: the character basis diagonalizes addition and yields a single pointwise operator, but has a finite unambiguous range; `digit  $\otimes$  place` does not diagonalize addition and needs sequential carries, but is exact and unbounded. Because a ring has no single basis diagonalizing both $+$ and $\times$ (the reason the main

method uses two bands), a joint representation or a cheap change-of-basis between operation-bases is the deeper open direction.

7. Discussion

The number code is appended alongside a model's existing word or byte dimensions, matching the intended "keep the existing dimensions for language, add mathematics in new ones" design. Several directions extend the result: (i) making the decode step differentiable end-to-end so the gate can be trained purely from answer supervision; (ii) handling magnitude beyond a fixed decode window, where the log code's monotonic phase provides a natural, order-preserving signal; and (iii) growing the fixed, shallow AST (a single binary operation) into **nested/parenthesised expressions** — since each operator returns a valid IR code, results compose directly ($M(18) \otimes M(9) \rightarrow M(162)$), so a parser feeding a tree of binds is the natural path — and rendering the final result back to natural language to close the NL $\rightarrow$ NL loop. Everything reported here is deterministic, self-contained, and reproducible on CPU in about a minute.

Limitations (stated plainly). The controlled result (§5) is scoped to $+$ and $-$, with **deterministic operand encoding** — numbers are fed as their spectral codes rather than parsed from raw text — and the learned component is a small op-word gate. So the claim is precisely "given the numbers already encoded, an *operator* built into the architecture extrapolates where a learned head does not," not "an end-to-end model learns to read and compute." The Fourier band also has a finite unambiguous range and an $O(\text{magnitude})$ matched-filter decode; $\times / \div$ were kept out of the differentiable test because their answer range makes a soft decode costly. The mechanism is prior art (HRR/VSA binding, the log-number system, grokking-Fourier); the framing, the controlled ablation, and the "right factors" analysis are the contribution.

The single next step that would make this a paper. Drop the operator layer, as a *differentiable* module, into a small end-to-end transformer that reads the raw string " $\langle a \rangle$ word $\langle b \rangle$ " character by character (so operand **parsing is learned**, not given), trained from answer supervision only, and compare against (a) the identical transformer with the operator block ablated and (b) a strong learned baseline that emits the answer digit by digit. If the model with the operator as an inductive bias extrapolates in magnitude where the identical model without it does not — with parsing learned — the architectural claim is established in full generality rather than in the controlled setting reported here. This subsumes the differentiable-decode and nested-expression directions above and is the natural path from "controlled result" to "credible paper."

Appendix: alternatives considered

"Comply with the Kronecker factorization" means finding a representation $\text{entity} \otimes \text{basis}$ in which the operation acts simply on the factors. Several candidates were weighed before choosing the Fourier/character route; they differ mainly in *which* operation each basis makes componentwise, and in their range.

method	$+$	$\times$	complies with Kronecker how	tradeoff
Residue Number System (CRT)	componentwise $\text{mod } p_i$	componentwise $\text{mod } p_i$	the <i>most literal</i> — $\mathbb{Z}/M \cong \otimes_i \mathbb{Z}/p_i$ is a ring tensor	needs a second per-prime code so $\times$ is a rotation too (discrete log);

method	+	×	complies with Kronecker how	tradeoff
			factorization; both ops componentwise	bounded (mod M)
Polynomial / digit-DFT	coeff. add + carry	convolution → pointwise in the DFT domain (convolution theorem; FFT multiplication)	keeps digit $\otimes$ place , and the DFT over <i>place</i> makes multiplication pointwise	still needs carry; two-stage
Fourier / character (used here)	phase rotation $e^{i\omega n}$	log-Fourier / Mellin $n^{i\omega}$	operation-diagonalizing basis; each op becomes a pointwise product	needs two bands (+ and × don't share a basis); bounded range
Irreducible-representation / matrix binding	1-D irreps on the unit circle (= Fourier)	—	the general lens: factors are the operation's irreps; non-abelian ops give matrix-valued factors and binding becomes matrix multiplication	heavier machinery; only needed for non-commutative operations
Learned bilinear / tensor operator	learn T with $\text{Rep}(a+b) = T(\text{Rep}(a), \text{Rep}(b))$	same	a bilinear map is itself a Kronecker-structured 3-tensor; connects to "grokking" (Fourier features emerge)	may not generalize unless the tensor is constrained

The Residue Number System is the tightest single-factorization fit (both operations componentwise), and the digit-DFT is the most elegant route for multiplication while retaining the **digit** $\otimes$ **place** structure. This work chooses the Fourier/character basis because it makes each operation an exact, parameter-free *pointwise* product and is the canonical basis in which addition diagonalizes (§2, §6). There is no free lunch: the right factorization is chosen relative to the operation and the numeric range required.

Related work (honest scoping)

The *mechanism* here is not new. Encoding a number as unit phasors and binding by an elementwise complex product is **Holographic Reduced Representations / Vector Symbolic Architectures** (Plate 1995; Kanerva 2009); doing multiplication in the log domain is the classical **log-number system**; and Fourier/"clock" number features are known to *emerge* in trained networks that learn modular arithmetic (**grokking**; Nanda et al. 2023). **NAC/NALU** (Trask et al. 2018) is the standard learned module for arithmetic extrapolation, and is used here as a baseline. What this submission contributes is not the encoding but (i) the **framing** — a mathematical IR with explicit operators, under a compiler analogy; (ii) a **controlled result** that the *operator-as-architecture*, not the

features, is what extrapolates (§5); and (iii) the **representation-theoretic "right factors" analysis** (§6). These are claims about framing and evidence, honestly scoped, not a new primitive.

All results are produced by the accompanying code (`experiments/exactness.py` , `experiments/end2end.py` , `experiments/operator_layer.py` , `experiments/factorization.py`); the interactive companion runs the same computation live in the browser.